

MazeGenerator — pełne omówienie kodu i przygotowanie do odpowiedzi

Dokument pomocniczy do obrony projektu: struktura kodu, klasy, metody, algorytmy, zależności i typowe pytania.

Jak korzystać z dokumentu

Najpierw przeczytaj sekcję o przepływie programu, potem klasy po kolei. Na końcu znajduje się lista pytań, które mogą paść podczas odpowiedzi z kodu.

1. Ogólny przepływ działania programu

Program składa się z części logicznej oraz części wizualnej. Część logiczna tworzy labirynt jako graf komórek, a część wizualna rysuje wynik na elemencie canvas w przeglądarce.

- Użytkownik wpisuje szerokość i wysokość labiryntu w interfejsie.
- Frontend tworzy obiekt `KruskalMaze` i wywołuje `generate()`.
- Generator buduje siatkę komórek, listę ścian, tasuje ściany Fisherem-Yatesem i usuwa wybrane ściany algorytmem Kruskala.
- Po wygenerowaniu labiryntu użytkownik może uruchomić A*, Dijkstrę lub porównanie obu algorytmów.
- Solver zwraca ścieżkę oraz kolejność odwiedzonych komórek.
- `CanvasRenderer` rysuje odwiedzone komórki, znaną ścieżkę, start, koniec i ściany.
- Frontend może pobrać aktualne porównanie jako plik PNG.

```
index.html → main.ts → KruskalMaze.generate() → Solver.solve() → CanvasRenderer.draw()
```

2. Struktura folderów

Projekt jest podzielony na foldery zgodne z odpowiedzialnościami klas. Taki podział ułatwia obronę, bo można wyjaśniać projekt warstwowo.

Folder	Znaczenie
src/core	Podstawowe modele: <code>Cell</code> i <code>Wall</code> .
src/structures	Struktury danych: <code>DisjointSet</code> i <code>MinHeap</code> .
src/maze	Klasa bazowa <code>Maze</code> i generator <code>KruskalMaze</code> .
src/solvers	<code>Solver</code> , <code>AStarSolver</code> i <code>DijkstraSolver</code> .
src/render	<code>CanvasRenderer</code> , czyli rysowanie labiryntu.
src/frontend	<code>main.ts</code> i <code>style.css</code> , czyli interfejs użytkownika.
src/errors	Własne klasy błędów.
.github/workflows	Automatyczny build i deploy na GitHub Pages.

3. core/Cell.ts

`Cell` reprezentuje pojedynczą komórkę labiryntu. Jest to podstawowy element całej aplikacji.

- `_x` i `_y` przechowują pozycję komórki w siatce.
- `_connections` to `Set`, czyli zbiór komórek, do których można przejść.
- `x()` i `y()` są getterami zgodnymi z przyjętym stylem OOP.
- `connections()` zwraca zbiór połączeń.

- `connect(other)` dodaje połączenie między komórkami. W praktyce oznacza usunięcie ściany między nimi.

Ważne na odpowiedź

W tym projekcie nie przechowujemy ścian jako `top/right/bottom/left` w komórce. Zamiast tego komórka przechowuje połączenia z sąsiadami. Brak połączenia oznacza ścianę.

4. core/Wall.ts

Wall przechowuje dwie sąsiednie komórki: `_a` i `_b`. Ściana jest potencjalną barierą między tymi komórkami.

- `a()` zwraca pierwszą komórkę ściany.
- `b()` zwraca drugą komórkę ściany.
- Wall sama nie usuwa ściany — decyzję podejmuje `KruskalMaze`.
- Jeśli `Kruskal` decyduje o usunięciu ściany, wykonywane jest połączenie komórek przez `connect()`.

5. structures/DisjointSet.ts

`DisjointSet`, czyli `Union-Find`, służy do obsługi rozłącznych zbiorów. Jest używany podczas generowania labiryntu `Kruskałem`.

- `_parent`: Map przechowuje rodzica każdego elementu.
- `_rank`: Map pomaga ograniczać wysokość drzew.
- `makeSet(x)` tworzy jednoelementowy zbiór.
- `find(x)` znajduje reprezentanta zbioru i wykonuje kompresję ścieżki.
- `union(a, b)` łączy dwa zbiory i zwraca `true`, jeśli połączenie faktycznie nastąpiło.

Kompresja ścieżki w `find()`

Jeżeli element wskazuje na rodzica, a rodzic na kolejnego rodzica, metoda `find()` ustawia element bezpośrednio na korzeń. Dzięki temu kolejne zapytania są szybsze.

Union by rank

Przy łączeniu zbiorów drzewo o mniejszej randze jest podpinane pod drzewo o większej randze. Jeżeli rangi są równe, jedno drzewo zostaje korzeniem, a jego ranga rośnie o 1.

```
if union(a, b) == true:
    a.connect(b)
else:
    #ściana zostaje, bo jej usunięcie utworzyłoby cykl
```

6. structures/MinHeap.ts

`MinHeap` to kolejka priorytetowa. `A*` i `Dijkstra` potrzebują struktury, która szybko zwróci komórkę o najmniejszym priorytecie.

- `push(item, priority)` dodaje element z priorytetem.
- `pop()` usuwa i zwraca element o najmniejszym priorytecie.
- `_siftUp()` przesuwaa element w górę kopca po dodaniu.
- `_siftDown()` przywraca własność kopca po usunięciu korzenia.
- `size()` zwraca liczbę elementów.

W `Dijkstrze` priorytetem jest odległość od startu. W `A*` priorytetem jest $fScore = gScore + heuristic$.

7. maze/Maze.ts

`Maze` jest klasą abstrakcyjną. Nie tworzy się jej bezpośrednio, tylko dziedziczy po niej `KruskalMaze`.

- `_width` i `_height` przechowują rozmiar labiryntu.
- `_grid`: `Cell[][]` przechowuje siatkę komórek w układzie `_grid[y][x]`.
- `cell(x, y)` zwraca komórkę o podanych współrzędnych.
- `_initGrid()` tworzy nową siatkę komórek.
- `_buildWalls()` tworzy listę ścian między sąsiednimi komórkami.
- `_shuffle()` tasuje tablicę algorytmem Fishera-Yatesa.
- `generate()` jest abstrakcyjne i musi zostać zaimplementowane w klasie pochodnej.

Dlaczego `_grid[y][x]`?

Pierwszy indeks oznacza wiersz, czyli `y`. Drugi indeks oznacza kolumnę, czyli `x`. To jest typowy układ dla tablic dwuwymiarowych reprezentujących siatkę.

`_buildWalls()`

Metoda dodaje ściany tylko w prawo i w dół. Dzięki temu każda ściana pojawia się w liście dokładnie raz, a nie podwójnie.

```
dla każdej komórki (x, y):
jeśli istnieje sąsiad z prawej → dodaj Wall(current, right)
jeśli istnieje sąsiad z dołu → dodaj Wall(current, bottom)
```

8. `maze/KruskalMaze.ts`

`KruskalMaze` jest konkretną implementacją `Maze`. Jej zadaniem jest wygenerowanie labiryntu przez usuwanie odpowiednich ścian.

- Na początku `generate()` inicjalizuje siatkę przez `_initGrid()`.
- Potem buduje listę ścian przez `_buildWalls()`.
- Następnie tasuje listę ścian przez `_shuffle()`.
- Dla każdej komórki wykonuje `makeSet()`.
- Dla każdej ściany sprawdza `union(a, b)`.
- Jeśli `union` zwróci `true`, komórki są łączone przez `a.connect(b)`.

Kluczowa odpowiedź
Kruskal nie usuwa każdej ściany. Usuwa tylko te ściany, które łączą dwie różne składowe. Dzięki temu labirynt pozostaje spójny i nie tworzą się cykle.

9. `solvers/Solver.ts`

`Solver` jest abstrakcyjną klasą bazową dla algorytmów rozwiązujących labirynt.

- `_maze` przechowuje referencję do labiryntu.
- `solve(start, goal)` jest metodą abstrakcyjną, implementowaną w `AStarSolver` i `DijkstraSolver`.
- `_reconstruct(came, end)` odtwarza ścieżkę od końca do startu na podstawie mapy poprzedników.

Rekonstrukcja działa od celu do startu: bierzemy `end`, potem `came.get(end)`, potem poprzednika poprzednika itd. Na końcu odwracamy tablicę, aby dostać kolejność od startu do celu.

10. `solvers/DijkstraSolver.ts`

`Dijkstra` znajduje najkrótszą ścieżkę w grafie z nieujemnymi wagami. W projekcie każdy ruch między komórkami ma koszt 1.

- `distances` przechowuje najlepszy znany dystans od startu do komórki.
- `came` przechowuje poprzednika każdej komórki na najlepszej ścieżce.
- `visited` zapobiega ponownemu przetwarzaniu komórek.
- `heap` wybiera komórkę o najmniejszym dystansie.

- `_visitedCount` liczy odwiedzone komórki.
- `visitedOrder` zapisuje kolejność odwiedzania do animacji.

```
start distance = 0
wrzuć start do heap
dopóki heap nie jest pusty:
  current = pop najmniejszego dystansu
  jeśli current == goal → odwróć ścieżkę
  dla każdego sąsiada:
    jeśli nowy dystans jest lepszy → aktualizuj distances, came i heap
```

11. solvers/AStarSolver.ts

A* działa podobnie do Dijkstry, ale używa heurystyki, która kieruje wyszukiwanie w stronę celu.

- `gScore` oznacza koszt od startu do bieżącej komórki.
- `heuristic` oznacza szacowaną odległość do celu.
- `fScore` = `gScore` + `heuristic` jest priorytetem w kolejce.
- Heurystyka Manhattan pasuje do ruchu po siatce w czterech kierunkach.
- A* zwykle odwiedza mniej komórek niż Dijkstra.

```
heuristic(a, b) = |a.x - b.x| + |a.y - b.y|
fScore = gScore + heuristic
```

Dlaczego A* jest poprawny?

Heurystyka Manhattan nie przeszacowuje kosztu w siatce z ruchami góra/dół/lewo/prawo i kosztem 1, więc A* nadal znajduje najkrótszą ścieżkę.

12. render/CanvasRenderer.ts

CanvasRenderer odpowiada tylko za rysowanie. Nie generuje labiryntu i nie rozwiązuje labiryntu — dostaje gotowe dane i pokazuje je na canvasie.

- `draw(maze, path, visited)` czyści canvas i rysuje wszystko od nowa.
- Najpierw rysowane jest białe tło.
- Potem rysowane są odwiedzone komórki kolorem żółtym.
- Następnie rysowana jest finalna ścieżka kolorem pomarańczowym.
- Start jest zielony, koniec czerwony.
- Na końcu rysowane są ściany, żeby były widoczne na wierzchu.

Ściany są wyznaczane na podstawie braku połączenia między sąsiednimi komórkami. Jeśli `cell.connections().has(neighbor)` jest `false`, renderer rysuje ścianę między nimi.

13. frontend/main.ts

`main.ts` łączy logikę algorytmów z interfejsem użytkownika. Pobiera elementy HTML, obsługuje przyciski i uruchamia animację.

- `createMaze()` tworzy nowy labirynt i inicjalizuje dwa renderery: dla A* i Dijkstry.
- `solveAStar()` uruchamia solver A* i zwraca `path`, `visitedOrder`, `visited` oraz `time`.
- `solveDijkstra()` robi analogicznie dla Dijkstry.
- `animateSingle()` animuje jeden algorytm na jednym canvasie.
- `animateBoth()` animuje oba algorytmy równolegle.
- `downloadComparison()` tworzy jeden obraz PNG z obu canvasów.

animationToken

`animationToken` zabezpiecza przed sytuacją, w której użytkownik zacznie nową animację lub wygeneruje nowy labirynt w trakcie poprzedniej animacji. Każda animacja dostaje swój token. Jeśli

token przestaje się zgadzać, animacja przerywa działanie.

Dlaczego są dwa canvasy?

Dwa canvasy pozwalają porównać A* i Dijkstrę obok siebie. Dzięki temu widać nie tylko wynik końcowy, ale też różnicę w sposobie przeszukiwania grafu.

14. errors

Folder errors zawiera własne klasy błędów. Dzięki temu błędy są bardziej opisowe niż zwykłe Error.

- MazeError jest klasą bazową dla błędów projektu.
- InvalidDimensionsError informuje o niepoprawnym rozmiarze labiryntu.
- CellOutOfBoundsError informuje o próbie pobrania komórki spoza siatki.
- NoPathFoundError informuje, że solver nie znalazł ścieżki.

15. GitHub Pages i Vite

Projekt jest uruchamiany lokalnie przez Vite. GitHub Pages nie uruchamia TypeScriptu bezpośrednio, dlatego potrzebny jest build do folderu dist.

- vite.config.ts ustawia base: /MazeGenerator/, bo strona działa pod adresem z nazwą repozytorium.
- npm run build tworzy produkcyjną wersję w dist.
- GitHub Actions wykonuje npm ci, npm run build i publikuje dist na Pages.
- .gitignore ignoruje node_modules, dist, .vscode, .env i .DS_Store.

16. Typowe pytania na odpowiedzi z kodu

Dlaczego A* i Dijkstra zwracają tę samą ścieżkę?

Bo labirynt generowany Kruskalem jest perfect maze, czyli między startem i końcem istnieje dokładnie jedna ścieżka. Oba algorytmy znajdują najkrótszą trasę, więc wynik długościowo jest taki sam.

Po co jest Fisher-Yates?

Żeby losowo przetasować ściany przed Kruskalem. Dzięki temu labirynt nie jest deterministyczny i za każdym razem może wyglądać inaczej.

Po co DisjointSet?

Żeby szybko sprawdzić, czy dwie komórki są już w tej samej składowej. Jeśli są, usunięcie ściany utworzyłoby cykl.

Dlaczego użyto MinHeap?

A* i Dijkstra muszą często wybierać komórkę o najmniejszym priorytecie. MinHeap robi to efektywniej niż liniowe przeszukiwanie tablicy.

Co oznacza visitedOrder?

To lista komórek w kolejności ich przetwarzania przez solver. Frontend używa jej do animacji.

Dlaczego renderer rysuje ściany na końcu?

Żeby ściany były widoczne nad kolorami odwiedzonych komórek i ścieżki.

Czy projekt jest obiektowy?

Tak. Każda klasa ma konkretną odpowiedzialność: Cell, Wall, Maze, KruskalMaze, Solver, AStarSolver, DijkstraSolver, CanvasRenderer, DisjointSet, MinHeap.

Co oznaczają pola z podkreśleniem?

To przyjęta konwencja projektu dla pól prywatnych lub chronionych, np. `_grid`, `_width`, `_visitedOrder`.

Dlaczego Maze jest abstract?

Bo Maze opisuje wspólne zachowanie labiryntu, ale nie definiuje konkretnego algorytmu generowania. Konkretna generacja jest w `KruskalMaze`.

Czy można dodać inny generator?

Tak. Wystarczy stworzyć klasę dziedziczącą po `Maze` i zaimplementować `generate()`, np. `RecursiveBacktrackerMaze`.

17. Krótka ściągą do odpowiedzi

Generacja:

`Maze._initGrid()` → `Maze._buildWalls()` → `Maze._shuffle()` → `DisjointSet.makeSet()` → `union()` → `connect()`

Solvency:

`start + goal` → `MinHeap` → `visitedOrder` → `came map` → `_reconstruct()` → `path`

Render:

`visited cells` → `path` → `start/end` → `walls`

Najważniejsze zdanie do zapamiętania

Program modeluje labirynt jako graf komórek, generuje perfect maze algorytmem Kruskala z tasowaniem Fishera-Yatesa, a następnie porównuje A* i Dijkstrę przez ścieżkę, liczbę odwiedzonych komórek i animację kolejności przeszukiwania.